CO-4 AT-1 CODE CHALLENGE

CSA0603-DAA

FRACTIONAL KNAPSACK

Greedy Algorithm Approach

Submitted By

BUSIREDDY CHAITANYA PRAKASH REDDY

Register Number: 192425255

Q6. Fractional Knapsack

A trader needs to load goods into a container with limited capacity. Unlike 0/1 Knapsack, items can be divided into smaller parts, allowing fractional selection. To maximize profit, items should be selected based on their value-to-weight ratio using a Greedy approach.

1. ABSTRACT

The Fractional Knapsack problem is a classical optimization problem used to demonstrate the Greedy algorithm. The problem consists of a collection of items, where each item has a value and a weight, and a container with a fixed weight capacity. Unlike the 0/1 Knapsack problem, the Fractional Knapsack problem allows an item to be divided into smaller portions. Therefore, an item does not have to be selected completely. A fraction of an item can be selected when the remaining capacity is insufficient for the complete item.

The main objective is to maximize the total value placed inside the container without exceeding its capacity. The greedy strategy used for this problem is to calculate the value-to-weight ratio for every item and select items in decreasing order of this ratio. An item with a higher ratio provides more value for every unit of weight, making it the most profitable choice at that stage.

This report explains the problem definition, objectives, greedy strategy, algorithm, pseudocode, sample calculation, implementation, complexity analysis, advantages, limitations, applications, and result. For the given sample input with values 60, 100, 120, weights 10, 20, 30, and capacity 50, the maximum obtainable profit is 240.

2. INTRODUCTION

Knapsack problems are important optimization problems in computer science and operations research. They are commonly used when limited resources must be allocated among different choices. Each item in a knapsack problem has a weight and a value, while the knapsack has a maximum capacity. The challenge is to decide which items should be included so that the total value is as large as possible.

In the Fractional Knapsack problem, splitting is allowed. This special property makes a greedy solution possible. The algorithm repeatedly chooses the item with the highest value per unit weight. If there is not enough capacity for the entire item, only the fraction that fits is selected.

3. PROBLEM STATEMENT

A trader needs to load goods into a container with limited capacity. Each good has a known value and weight. The trader wants to maximize the total value of the goods loaded into the container. Unlike the 0/1 Knapsack problem, goods can be divided into smaller parts, so fractional quantities may be selected.

4. OBJECTIVES

• To understand the Fractional Knapsack optimization problem.

• To apply the Greedy algorithm to obtain the maximum possible profit.

• To calculate the value-to-weight ratio for every item.

• To select complete or fractional items according to the remaining capacity.

• To analyze the time and space requirements of the solution.

• To implement the algorithm in C programming.

5. BASIC CONCEPT

Suppose there are n items. For item i, let value[i] represent its value and weight[i] represent its weight. Let W be the capacity of the knapsack. The profit obtained from one unit of weight of an item is represented by:

Ratio[i] = Value[i] / Weight[i]

The item with the largest ratio is considered first. This continues until the knapsack is full. If an item is too heavy to fit completely, the algorithm takes only the fraction that can fit.

6. DIFFERENCE FROM 0/1 KNAPSACK

In 0/1 Knapsack, an item can either be completely selected or completely rejected. It cannot be split. In Fractional Knapsack, an item can be divided, which means 10 kg of a 30 kg item can be selected if only 10 kg of capacity remains. Because of this property, the greedy ratio strategy produces an optimal solution for Fractional Knapsack.

7. GREEDY APPROACH

The Greedy approach makes the best available choice at every step. For Fractional Knapsack, the best choice is the item with the maximum value-to-weight ratio. This strategy is appropriate because every fraction of an item has the same value-to-weight ratio as the complete item.

8. ALGORITHM

1. Start with total profit equal to zero.
2. Read the number of items, their values, weights, and the knapsack capacity.
3. Calculate value/weight ratio for each item.
4. Sort the items in descending order of their ratios.
5. Start from the item with the highest ratio.
6. If the complete item fits into the remaining capacity, select it completely.
7. Otherwise, select only the fraction that fits into the remaining capacity.
8. Add the corresponding profit to the total profit.
9. Stop when the knapsack becomes full or all items have been considered.

10. Display the maximum profit.

9. CODE

```
#include <stdio.h>

int main() {
    int n, W;
    float value[100], weight[100], ratio[100];
    float maxProfit = 0;
    printf("Enter number of items: ");
    scanf("%d", &n);
    printf("Enter values:\n");
    for (int i = 0; i < n; i++)
        scanf("%f", &value[i]);
    printf("Enter weights:\n");
    for (int i = 0; i < n; i++)
        scanf("%f", &weight[i]);
    printf("Enter capacity: ");
    scanf("%d", &W);
    for (int i = 0; i < n; i++)
        ratio[i] = value[i] / weight[i];
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (ratio[i] < ratio[j]) {
                float temp;
                temp = ratio[i];
                ratio[i] = ratio[j];
```

```
                ratio[j] = temp;
                temp = value[i];
                value[i] = value[j];
                value[j] = temp;
                temp = weight[i];
                weight[i] = weight[j];
                weight[j] = temp;
            }
        }
    }
    for (int i = 0; i < n; i++) {
        if (W >= weight[i]) {
            W -= weight[i];
            maxProfit += value[i];
        } else {
            maxProfit += ratio[i] * W;
            break;
        }
    }
    printf("Maximum Profit = %.0f\n", maxProfit);
    return 0;
}
```

10. FLOW OF THE SOLUTION

Input → Calculate value/weight ratios → Sort by ratio → Select highest ratio item → Check remaining capacity → Take full item or required fraction → Update profit and capacity → Repeat → Display maximum profit.

11. SAMPLE INPUT

n = 3

Values = 60 100 120

Weights = 10 20 30

W = 50

12. SAMPLE CALCULATION

Item	Value	Weight	Value/Weight
1	60	10	6
2	100	20	5
3	120	30	4

The ratios are 6, 5, and 4 respectively. Therefore, Item 1 is selected first, followed by Item 2, and then a fraction of Item 3.

Step 1: Item 1 is completely selected. Weight used = 10, profit = 60.

Step 2: Item 2 is completely selected. Weight used = 20, profit = 100.

Remaining capacity = 50 − 10 − 20 = 20.

Step 3: Item 3 has weight 30, but only 20 capacity remains.

Fraction selected = 20/30.

Profit from Item 3 = 120 × (20/30) = 80.

Total maximum profit = 60 + 100 + 80 = 240.

13. EXPECTED OUTPUT

Maximum Profit = 240

```
Enter number of items: 3
Enter values:
60 100 120
Enter weights:
10 20 30
Enter capacity: 50

+------+-------+--------+--------------+
| Item | Value | Weight | Value/Weight |
+------+-------+--------+--------------+
| 1    | 60    | 10     | 6.00         |
| 2    | 100   | 20     | 5.00         |
| 3    | 120   | 30     | 4.00         |
+------+-------+--------+--------------+

Maximum Profit = 240
```

14. TIME AND SPACE COMPLEXITY

The implementation uses a simple comparison-based sorting method. The sorting stage takes $O(n^2)$ time. The selection stage takes $O(n)$ time. Therefore, the overall time complexity of this particular implementation is $O(n^2)$. The arrays used to store values, weights, and ratios require $O(n)$ space.

16. ADVANTAGES

• Simple and easy to understand.

• Efficient for the Fractional Knapsack problem.

• Allows partial selection of items.

• Provides the optimal solution when fractional selection is allowed.

• Greedy decisions reduce the need to examine every possible combination.

17. LIMITATIONS

• The greedy method is not generally optimal for the 0/1 Knapsack problem.

• The implementation's simple sorting method has $O(n^2)$ time complexity.

• Items must be divisible for the fractional strategy to be valid.

• Floating-point calculations may require care when exact numerical precision is important.

18. RESULT

The Fractional Knapsack problem was successfully solved using the Greedy approach. The items were ranked according to their value-to-weight ratios, and items were selected in descending order. For the given sample, the first two items were completely selected and 20 units of the third item were selected from its total weight of 30 units. The resulting maximum profit is 240.

19. CONCLUSION

The Fractional Knapsack problem demonstrates how a suitable greedy strategy can solve an optimization problem efficiently. By choosing the item with the highest value per unit weight, the algorithm makes locally optimal decisions that lead to a globally optimal solution when items are divisible. The problem is an important example for understanding greedy algorithms, sorting, optimization, and algorithm complexity.